

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1707

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 35%

QUESTIONS: 3

Duration : 1 Hour
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I KISHORE KUMAR V(192521371)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

KISHORE V

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

- i. D1 cannot work Night shift
- ii. D2 must work before D3 (Morning < Afternoon < Night)
- iii. Only one doctor per shift
- iv. D3 cannot work Morning
- v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- i. Movement allowed: Up, Down, Left, Right
- ii. Cost of each move = 1
- iii. Cannot pass through obstacles
- iv. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

- a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

- b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
- c) Demonstrate how the robot applies AI concepts such as:
- Intelligent agents
 - Rationality
 - Environment type
- d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

ANSWERS

Question 1: Hospital Shift Scheduling (Backtracking Search)

1.1 Problem formulation (CSP)

- **Variables:** D1, D2, D3 (each doctor's shift)
- **Domain:** {Morning (M), Afternoon (A), Night (N)} with order $M < A < N$
- **Constraints:**
 C1: $D1 \neq \text{Night}$. C2: $\text{shift}(D2) < \text{shift}(D3)$. C3: all shifts different (one doctor per shift). C4: $D3 \neq \text{Morning}$. C5: every doctor gets exactly one shift (complete assignment).
- **Search settings:** variable order $D1 \rightarrow D2 \rightarrow D3$, value order $M \rightarrow A \rightarrow N$. A constraint is checked as soon as all of its variables are assigned.

1.2 Backtracking trace with constraint checks

Step	Assignment tried	Constraint checks	Result
1	D1 = M	C1: $M \neq N \checkmark$. C3: no clash $\checkmark$	Continue to D2
2	D1 = M, D2 = M	C3: Morning already used by D1 $\times$	Reject, try next value
3	D1 = M, D2 = A	C3 $\checkmark$ (M, A distinct). C2 not yet testable (D3 unassigned)	Continue to D3
4	D1 = M, D2 = A, D3 = M	C3: Morning used $\times$. C4: D3 = Morning $\times$	Reject
5	D1 = M, D2 = A, D3 = A	C3: Afternoon used by D2 $\times$	Reject
6	D1 = M, D2 = A, D3 = N	C1 $\checkmark$. C2: $A < N \checkmark$. C3 $\checkmark$ (M, A, N). C4: $N \neq M \checkmark$. C5 $\checkmark$ all assigned	SOLUTION FOUND

Continuing the search for completeness (to see whether other schedules exist):

Step	Assignment tried	Constraint checks	Result
7	D1 = M, D2 = N	C3 $\checkmark$. Continue to D3	Continue to D3
8	... D3 = M / A / N	D3 = M: C3 $\times$, C4 $\times$. D3 = A: C2: $N < A$ is false $\times$. D3 = N: C3 $\times$ (Night used)	All fail $\rightarrow$ backtrack to D1
9	D1 = A, D2 = M	C1 $\checkmark$, C3 $\checkmark$. D3 = M $\times$ (C3, C4); D3 = A $\times$ (C3); D3 = N: C2: $M < N \checkmark$, C3 $\checkmark$, C4 $\checkmark$	Second solution
10	D1 = A, D2 = A / N	D2 = A: C3 $\times$. D2 = N: D3 = M $\times$, A $\times$ (C3 / C2), N $\times$	Dead end $\rightarrow$ backtrack
11	D1 = N	C1: D1 cannot work Night $\times$	Pruned, search ends

1.3 Final valid schedule

Morning	Afternoon	Night
D1	D2	D3

Result: D1 $\rightarrow$ Morning, D2 $\rightarrow$ Afternoon, D3 $\rightarrow$ Night (first solution found by backtracking). All five constraints hold: D1 is not on Night, D2 (Afternoon) works before D3 (Night), each shift has one doctor, D3 is not on Morning, and every doctor has exactly one shift.

Note: The full search also shows one other valid schedule ($D2 \rightarrow \text{Morning}$, $D1 \rightarrow \text{Afternoon}$, $D3 \rightarrow \text{Night}$). The constraints do not make the solution unique, so either one is acceptable; the first one found is reported as the answer.

Question 2: Robot Navigation in a 5×5 Grid (BFS / Heuristic Search)

2.1 Problem set-up

The question does not include the grid diagram, so the following standard 5×5 layout is assumed. Cells are written as (row, column) starting at (0,0) at the top-left.

	c0	c1	c2	c3	c4
r0	S				
r1		■		■	
r2		■			
r3				■	
r4					G

S = Start (0,0), G = Goal (4,4), ■ = obstacles at (1,1), (1,3), (2,1) and (3,3).

- Actions: Up, Down, Left, Right (successors are generated in this order).
- Step cost = 1 for every move; obstacles and cells outside the grid cannot be entered.
- Heuristic: Manhattan distance $h(r,c) = |r - 4| + |c - 4|$. It is admissible and consistent because every move changes the distance by at most 1 and costs 1.
- Start state $h(S) = 8$, so the optimal cost cannot be less than 8.

2.2 Part A: BFS (FIFO queue) step-by-step expansion

BFS expands nodes level by level using a FIFO queue. Since every step costs 1, the first time BFS reaches G it has found the cheapest path. Nodes are marked visited when generated.

#	Expand	Generated (new)	Queue after expansion
1	(0,0)	(1,0), (0,1)	(1,0), (0,1)
2	(1,0)	(2,0) [(1,1) obstacle]	(0,1), (2,0)
3	(0,1)	(0,2) [(1,1) obstacle]	(2,0), (0,2)
4	(2,0)	(3,0) [(2,1) obstacle]	(0,2), (3,0)
5	(0,2)	(1,2), (0,3)	(3,0), (1,2), (0,3)
6	(3,0)	(4,0), (3,1)	(1,2), (0,3), (4,0), (3,1)
7	(1,2)	(2,2) [(1,1), (1,3) obstacles]	(0,3), (4,0), (3,1), (2,2)
8	(0,3)	(0,4) [(1,3) obstacle]	(4,0), (3,1), (2,2), (0,4)
9	(4,0)	(4,1)	(3,1), (2,2), (0,4), (4,1)
10	(3,1)	(3,2) [(2,1) obstacle; (4,1) already seen]	(2,2), (0,4), (4,1), (3,2)
11	(2,2)	(2,3)	(0,4), (4,1), (3,2), (2,3)
12	(0,4)	(1,4)	(4,1), (3,2), (2,3), (1,4)
13	(4,1)	(4,2)	(3,2), (2,3), (1,4), (4,2)
14	(3,2)	none [(3,3) obstacle; (4,2) already seen]	(2,3), (1,4), (4,2)
15	(2,3)	(2,4) [(1,3), (3,3) obstacles]	(1,4), (4,2), (2,4)
16	(1,4)	none [(2,4) already seen]	(4,2), (2,4)
17	(4,2)	(4,3)	(2,4), (4,3)
18	(2,4)	(3,4)	(4,3), (3,4)

#	Expand	Generated (new)	Queue after expansion
19	(4,3)	(4,4) = GOAL	(3,4), (4,4) → goal reached

BFS needed 19 expansions. The path is traced back through the parent pointers.

2.3 Part B: Priority queue with Manhattan heuristic (A* search, $f = g + h$)

Because the question asks for priority-queue updates and the Manhattan heuristic, the informed version of the search is shown: the queue is ordered by $f(n) = g(n) + h(n)$; ties are broken by smaller h (closer to the goal), then FIFO. Each queue entry is written as (cell, $g+h=f$).

#	Pop (lowest f)	Generated / updated	Priority queue after the step
1	(0,0) $g=0, h=8, f=8$	(1,0): $1+7=8$; (0,1): $1+7=8$	(1,0):8, (0,1):8
2	(1,0) $f=8$	(2,0): $2+6=8$ [(1,1) blocked]	(2,0):8 [$h=6$], (0,1):8 [$h=7$]
3	(2,0) $f=8$	(3,0): $3+5=8$ [(2,1) blocked]	(3,0):8 [$h=5$], (0,1):8
4	(3,0) $f=8$	(4,0): $4+4=8$; (3,1): $4+4=8$	(4,0):8, (3,1):8, (0,1):8
5	(4,0) $f=8$	(4,1): $5+3=8$	(4,1):8 [$h=3$], (3,1):8, (0,1):8
6	(4,1) $f=8$	(4,2): $6+2=8$ [(3,1) already has lower or equal g]	(4,2):8 [$h=2$], (3,1):8, (0,1):8
7	(4,2) $f=8$	(4,3): $7+1=8$; (3,2): $7+3=10$	(4,3):8, (3,1):8, (0,1):8, (3,2):10
8	(4,3) $f=8$	(4,4): $8+0=8$ [(3,3) blocked]	(4,4):8, (3,1):8, (0,1):8, (3,2):10
9	(4,4) $f=8$	GOAL TEST passed	Stop

With the heuristic only 9 nodes were expanded, compared with 19 for blind BFS. The heuristic never overestimates, and f stays at 8 along the whole path, so the first goal popped is optimal.

2.4 Optimal path and cost

Path: (0,0) → (1,0) → (2,0) → (3,0) → (4,0) → (4,1) → (4,2) → (4,3) → (4,4)

Moves: Down, Down, Down, Down, Right, Right, Right, Right (8 moves).

Optimal cost = 8 ($8 \text{ moves} \times 1$), which equals the Manhattan lower bound, so no cheaper path can exist. Other 8-cost paths exist (for example through (0,2), (1,2), (2,2), (2,3), (2,4), (3,4)), but all have the same cost.

Question 3: Autonomous Rescue Robot (Partially Observable, Online Search)

(a) Analysis of constraints and their effect on decision-making

Constraint	What it means	Effect on decisions
Moves: up, down, left, right	Four actions per cell, so a discrete state space with branching factor ≤ 4	Search space is small and finite; no diagonal shortcuts
Blocked cells (obstacles)	Some actions are illegal in some states	Robot must discover walls and plan around them; the plan can become invalid at any time
Limited sensing (adjacent cells only)	Environment is partially observable; the full map is unknown in advance	Robot cannot plan once and for all. It must keep a belief/map, act, sense and re-plan (online search)
Cost: 1 per move, +2 for a risky zone (total 3)	Non-uniform step costs	Plain BFS is not optimal. Cost-sensitive search (UCS / A*) is required. A risky cell is worth entering only if the detour around it costs more than 2 extra moves
Minimise total cost and stay safe	Two goals: low cost and safety	Risk is built into the cost, so safety and cost are traded off in a single utility measure; known blocked cells are hard constraints
Online knowledge update	The knowledge base changes during execution	Plan $\rightarrow$ move one step $\rightarrow$ sense $\rightarrow$ update map $\rightarrow$ re-plan loop; the robot must also avoid dead-end loops by remembering visited cells

Priority of constraints: (1) hard constraints first: movement rules and obstacles (infeasible moves are never allowed); (2) safety/risk costs, since they strongly shape the route; (3) total path cost minimisation; (4) sensing limitation, which determines how and when information arrives and therefore how often the robot must re-plan.

(b) Solution approach: Online Uniform Cost Search with re-planning

Search strategy: Uniform Cost Search (Dijkstra) run repeatedly inside an online agent loop. The Manhattan distance can be added as an admissible heuristic (every move costs at least 1), which turns the planner into A* and reduces the nodes expanded while keeping optimality.

Step-by-step procedure:

- **Step 1: Initialise.** Map $M = \{S \text{ known}, G \text{ known location (survivor position), all other cells unknown}\}$. Current position = S , accumulated cost = 0. Unknown cells are treated optimistically as free (cost 1), which is called the free-space assumption.
- **Step 2: Sense.** Observe the four adjacent cells. Record each as free, blocked or risky and update M .
- **Step 3: Plan.** Run UCS (or A*) on the current map M from the current cell to G , using step cost $c = 1$ for normal cells and $c = 1 + 2 = 3$ for risky cells. Blocked cells are excluded. The frontier is a priority queue ordered by $g(n)$, the cheapest cumulative cost.
- **Step 4: Act.** Execute only the first move of the cheapest plan, then add its cost to the total.
- **Step 5: Update and re-plan.** After arriving, sense again. If a new obstacle or risky zone lies on the planned path, discard that plan and re-run UCS from the current cell; otherwise continue.
- **Step 6: Terminate.** Repeat Steps 2–5 until the robot reaches G (success) or UCS finds no path in M (report that the survivor is unreachable with the current knowledge, and optionally explore unknown frontier cells).

Pseudo-code:

```

map ← {S, G}; pos ← S; total ← 0
while pos ≠ G:
    percepts ← sense(adjacent cells of pos); map ← update(map, percepts)
    plan ← UCS(map, pos, G) using cost 1 (normal) / 3 (risky), blocked excluded
    if plan is empty: return FAILURE (no known safe route)
    pos ← move(first action of plan); total ← total + cost(that move)
return SUCCESS, total

```

Decision rule illustrated: if going straight through one risky cell costs $1 + 2 = 3$ for that step, while a safe detour needs 2 extra moves (cost 2 extra), the safe detour is cheaper by 1 and is chosen. If the detour needs 3 or more extra moves, the risky cell is cheaper (a tie occurs when the detour costs exactly 2 extra) and the robot passes through it.

(c) Application of AI concepts**Intelligent agent**

The robot is a utility-based, model-based agent with learning of its environment map. PEAS description:

Component	Description
Performance measure	Reach the survivor (G) with minimum total cost (moves + risk penalties), avoiding blocked and unsafe cells
Environment	Disaster-affected building represented as a grid of rooms, with obstacles and risky zones
Actuators	Move Up, Down, Left, Right
Sensors	Adjacent-cell sensors (detect blocked, free, risky) and position/odometry

Percept → internal model → search → action: each percept updates the internal map (the agent's model of the world), the planner uses this model to pick the action that minimises expected cost, and the action is executed. This corresponds to the sense–think–act cycle of an agent.

Rationality

A rational agent selects the action expected to maximise its performance measure given its percept sequence and built-in knowledge. Here the robot is rational because it (i) chooses the lowest-cost route based on everything it has sensed so far, (ii) never enters cells known to be blocked, (iii) enters risky cells only when the cheaper alternative is more costly, and (iv) re-plans when new percepts invalidate the current plan. Rationality is bounded by what the robot can perceive: it is not omniscient, so it may take a slightly longer route when information is revealed late, but its choices are always optimal for what it knows.

Environment type

Property	Type	Reason
Observability	Partially observable	Only adjacent cells can be sensed
Agents	Single agent	Only the rescue robot acts (survivors and debris are not agents)
Determinism	Deterministic (assumed)	A chosen move reaches its target cell unless blocked; sensing noise is ignored
Episodic / sequential	Sequential	Each move affects later costs and options
Static / dynamic	Static (semi-dynamic if rubble shifts)	The layout does not change while the robot decides, but it is revealed over time

Property	Type	Reason
Discrete / continuous	Discrete	Grid cells, four discrete actions
Known / unknown	Unknown (partially known)	Map is learned as the robot explores, so online search is required

(d) Justification of the chosen approach

- **Handles non-uniform costs:** BFS and DFS ignore step costs. UCS always expands the cheapest cumulative path first, so it is complete and optimal when risky cells cost 3 and normal cells cost 1.
- **Handles partial observability:** An offline planner would need the full map. The online sense–plan–act loop with re-planning uses only what has been observed and adapts when new obstacles appear, so the robot never relies on a stale plan.
- **Safety:** Risk is converted into a numeric penalty and blocked cells are removed from the search, so safe navigation is built into the cost model instead of being an afterthought.
- **Optimality and completeness:** Given its current knowledge the robot always follows a least-cost path. In a finite grid with positive step costs the search is complete, and it will find the survivor if any path exists.
- **Efficiency:** The grid is small (branching factor ≤ 4), so repeated UCS is cheap. Adding the admissible Manhattan heuristic (A^*) reduces expansions further without losing optimality.
- **Better than alternatives:** Greedy best-first ignores cost and may walk into costly risky zones; hill climbing can get stuck in dead ends; DFS may produce long or unsafe paths; pure offline planning fails because the map is unknown.

Conclusion: An online agent that repeatedly applies uniform cost search (or A^* with Manhattan distance) on its updated map is the most suitable approach, because it satisfies the hard movement and obstacle constraints, trades off cost and risk rationally, and copes with the limited sensing of the environment.